Design Pattern Report: Observer Pattern

Group 52
Nguyễn Đình Minh Huy (Student ID: 25125083)

July 20, 2026

Contents

1 Real-world Problem **2**

2 Naive Solution (Without Design Pattern) **2**

3 Problems with the Naive Solution **3**

4 Introduction to the Observer Pattern **3**

5 General Class Diagram **3**

6 Class Diagram for the Bank Account Problem **4**

7 Implementation of the Observer Pattern **5**

8 Pros and Cons **6**
8.1 Pros 6
8.2 Cons 7

9 Other Real-world Applications **7**
9.1 Web Development 7
9.2 Mobile Development 7

10 Observer and Mediator Fusion **7**
10.1 Case Study: Super Mario Game EventBus 8

11 Quiz **10**

12 References **11**

1 Real-world Problem

Consider a modern financial core banking system where transactions are performed on a `BankAccount`. When a deposit occurs, several peripheral systems need to be notified of the event immediately to execute their processes:

- `UI`: Updates the account balance display on the user's screen.
- `Email System`: Sends a transaction alert email to the customer.
- `Logger`: Writes transaction details to the security audit trail.
- `Fraud AI`: Scans transaction metadata to detect suspicious patterns.
- `Mobile App`: Delivers a push notification to the client's phone.
- `Analytics`: Records transaction metrics for business intelligence.

A customer or administrator must be able to register, enable, or disable these notification channels dynamically. The core challenge is establishing a system where a bank account can broadcast transaction updates to an arbitrary and dynamic set of observer systems without knowing their concrete classes or internal notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `BankAccount` class holds references to concrete notification system objects directly. Below is the C++ representation of this approach:

```cpp
// Refer to Observer/src/naive.cpp for full implementation details
class NaiveBankAccount {
private:
    double balance;
    UI ui;
    EmailSystem email;
    Logger logger;
    FraudDetector fraudDetector;
    MobileApp mobileApp;
    Analytics analytics;

public:
    NaiveBankAccount(double initialBalance) : balance(initialBalance)
    {}

    void deposit(double amount) {
        balance += amount;

        // Direct, coupled method invocations on concrete systems
        ui.update(balance);
        email.sendEmail(amount);
        logger.log(amount, balance);
        fraudDetector.scan(amount);
        mobileApp.pushNotification(amount);
        analytics.trackActivity("Deposit", amount);
    }
};
```

Listing 1: Naive Implementation of Bank Account Notifications

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveBankAccount` is tightly coupled to concrete classes like `UI`, `EmailSystem`, `Logger`, `FraudDetector`, `MobileApp`, and `Analytics`. It must know their names, structures, and exactly which update method to call on each.

2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new system (such as a compliance reporting system or a credit score monitoring engine), we must modify the core `NaiveBankAccount` class. This requires adding a new member field and updating the notification logic inside the `deposit` method.

3. **Violation of the Single Responsibility Principle (SRP):** The `BankAccount` class should focus solely on financial ledger operations (managing the balance and transactions). However, it is also burdened with orchestrating emails, logs, mobile push notifications, and security analysis.

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to ConcreteObservers and sends notifications.
- **ConcreteObserver:** Implements the Observer updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general Observer pattern structure:

- A `Subject` maintains a list of references to `Observer` objects.
- The `Subject` exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- `Observer` exposes the `update()` method interface.

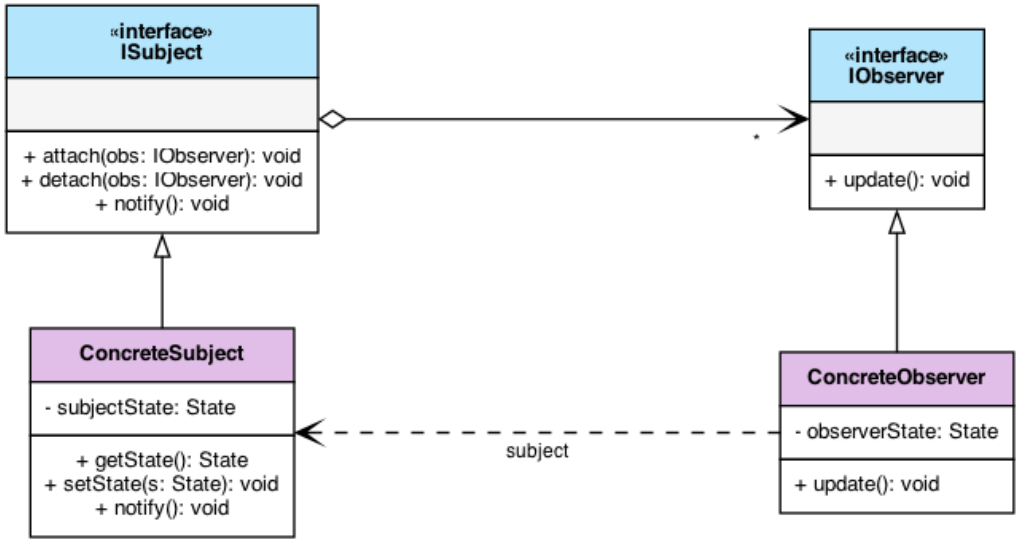

Figure 1: General Observer Design Pattern UML Diagram

6 Class Diagram for the Bank Account Problem

Applying the pattern to our problem:

- `BankAccount` acts as the ConcreteSubject (or subject coordinator).
- `AccountObserver` acts as the Observer interface.
- `UIObserver`, `EmailObserver`, `LoggerObserver`, `FraudAIObserver`, `MobileAppObserver`, and `AnalyticsObserver` act as the ConcreteObservers.

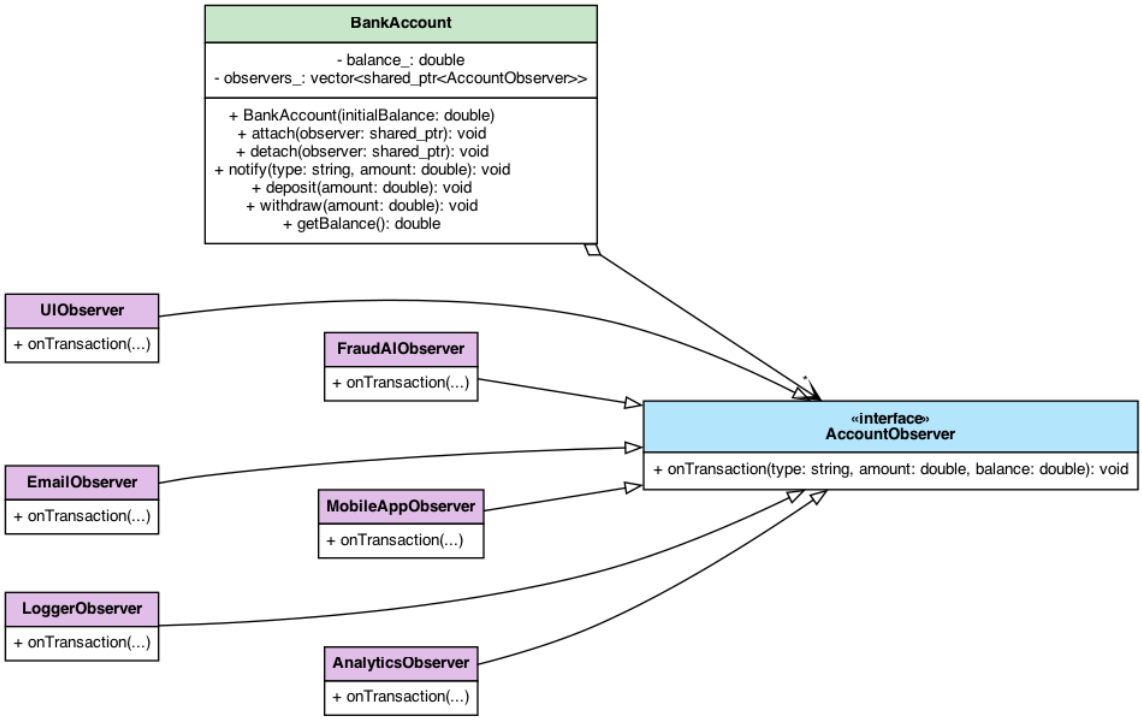

Figure 2: UML Class Diagram for Bank Account Notification System

7 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding or removing subscriber systems at runtime without modifying the core subject.

```cpp
// Refer to Observer/src/pattern.cpp for full implementation details
class AccountObserver {
public:
    virtual ~AccountObserver() = default;
    virtual void onTransaction(const std::string& type, double amount, double currentBalance) = 0;
};

class BankAccount {
private:
    double balance;
    std::vector<std::shared_ptr<AccountObserver>> observers;

public:
    BankAccount(double initialBalance) : balance(initialBalance) {}

    void attach(std::shared_ptr<AccountObserver> obs) { observers.push_back(obs); }
    void detach(std::shared_ptr<AccountObserver> obs) { /* Remove from list */ }
    void notifyObservers(const std::string& type, double amount) {
        for (const auto& obs : observers) {
            obs->onTransaction(type, amount, balance);
        }
    }

    void deposit(double amount) {
        balance += amount;
        notifyObservers("Deposit", amount);
    }
};
```

Listing 2: Refactored Observer Implementation

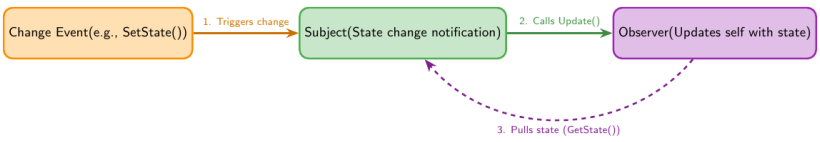

Figure 3: Sequential Process Flowchart of a Single Observer Notification

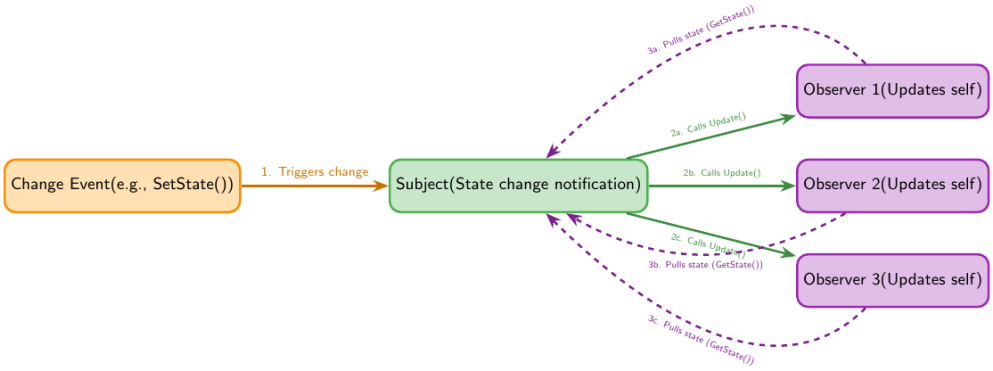

Figure 4: Sequential Process Flowchart with Multiple Observers (Pull Model)

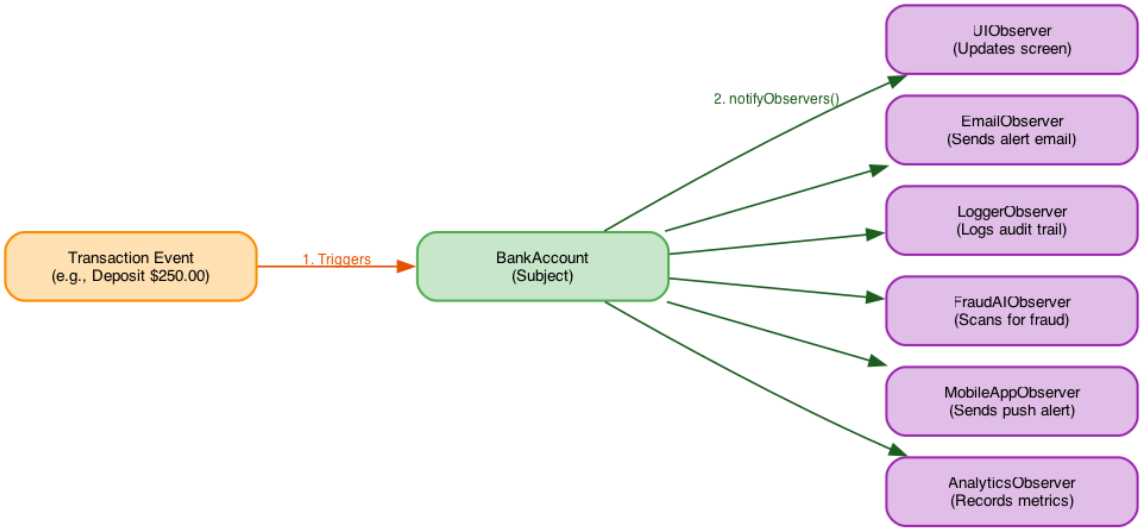

Figure 5: Process Flowchart for Bank Account Notification System (Push Model)

8 Pros and Cons

8.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered subscribers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

8.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.
- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.
- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

9 Other Real-world Applications

9.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.
- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

9.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

10 Observer and Mediator Fusion

In complex system architectures, the *Observer* and *Mediator* patterns are frequently fused together to handle coordination and communication between multiple components.

In a standard Mediator implementation, colleagues must explicitly call the mediator, and the mediator directly invokes actions on other colleagues. By fusing the two patterns:

- **Colleagues act as Subjects:** They maintain state and notify registered observers when changes occur, remaining completely unaware of who is listening.
- **The Mediator acts as an Observer:** It subscribes to the event streams of the Colleagues. When it receives an `update()` notification, it coordinates and executes the required reaction across other Colleagues.

This fusion achieves dynamic event-driven routing while preserving loose coupling between Colleagues.

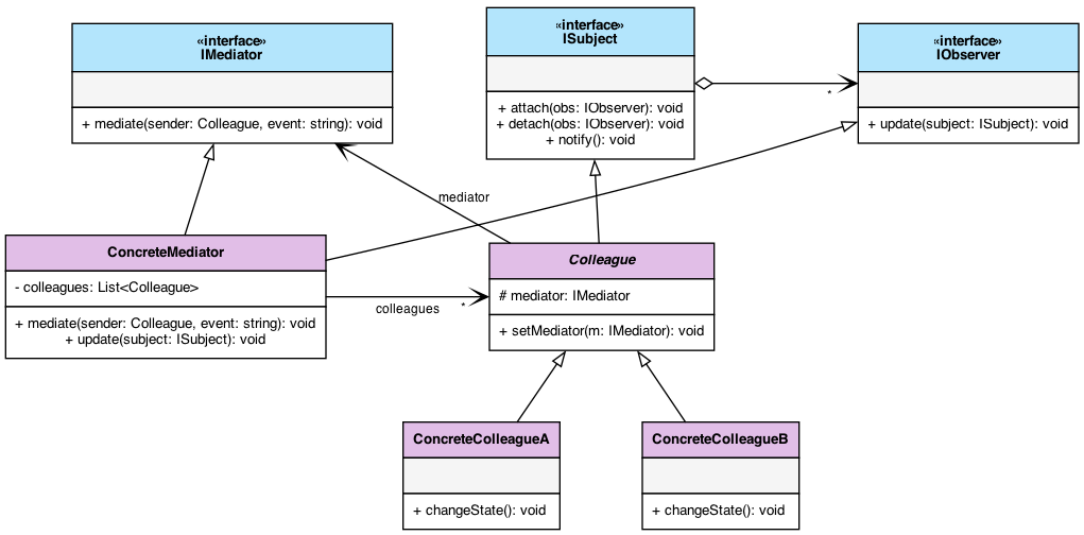

Figure 6: UML Class Diagram of the Observer-Mediator Fusion

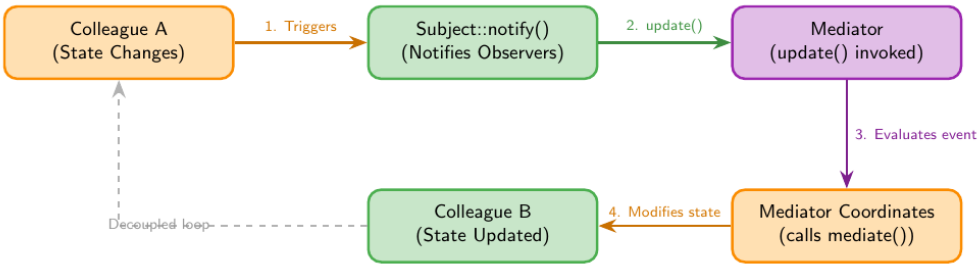

Figure 7: Process Flowchart of Event-Driven Mediation Flow

10.1 Case Study: Super Mario Game EventBus

In game development, handling interactions between numerous entities and systems (e.g., updating UI when a player dies, unlocking achievements when a coin is collected, playing sounds when blocks break) can easily lead to spaghetti code and tight coupling. To solve this, a decentralized event-routing system known as an **EventBus** is used.

The `EventBus` acts as a central singleton *Mediator*. Colleagues (such as the `Player` or `POWBlock`) publish events to the `EventBus` without knowing who reacts to them. Other systems (such as the `StatisticsTracker` or `AchievementManager`) register as *Observers* to specific event types.

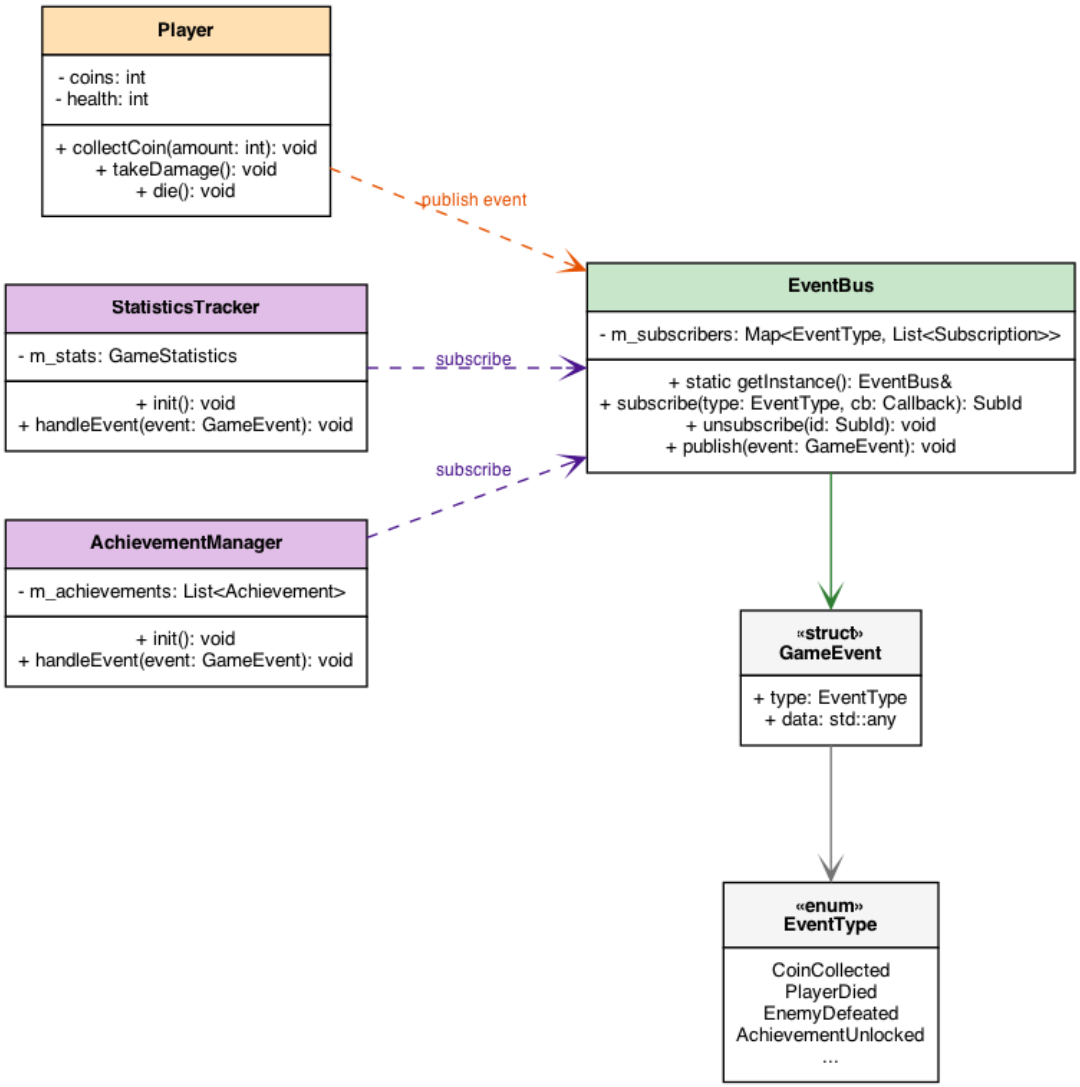

Figure 8: UML Class Diagram for Super Mario Game EventBus System

The C++ interface for the game's EventBus is implemented as follows:

```cpp
// include/Core/EventBus.hpp
class EventBus {
public:
  using Callback = std::function<void(const GameEvent &)>;
  using SubscriptionId = size_t;

  // Singleton Instance access
  static EventBus &getInstance();

  // Event operations (Mediating Hub)
  SubscriptionId subscribe(EventType type, Callback callback);
  void unsubscribe(SubscriptionId id);
  void publish(const GameEvent &event);

private:
  EventBus() = default;

  struct Subscription {
    SubscriptionId id;
```

```
    Callback callback;
  };

  std::unordered_map<EventType, std::vector<Subscription>>
   m_subscribers;
  SubscriptionId m_nextId = 0;
};
```

```
huynguyen@mario-engine: ~/eventbus_simulation

============================================================
          SUPER MARIO EVENTBUS SIMULATION DEMO
============================================================
[EventBus] System initialized. Singleton instance online.
[StatisticsTracker]  Subscribed to CoinCollected
[StatisticsTracker]  Subscribed to PlayerDied
[StatisticsTracker]  Subscribed to ComboHit
[AchievementManager]  Subscribed to CoinCollected
[AchievementManager]  Subscribed to AchievementUnlocked

>> Player collects 1 gold coin...
[Player] Collected coin! Publishing event...
[EventBus] Publishing: CoinCollected (data: 1)
[StatisticsTracker]  Received CoinCollected. Total coins: 1
[AchievementManager]  Received CoinCollected. Evaluating achievements...

>> Player hits a POW Block!
[POWBlock] Broken! Triggering screen shake and screen enemy flip...
[EventBus] Publishing: BlockBroken (data: POWBlock*)
[EventBus] (No subscribers registered for EventType::BlockBroken)

>> Player defeats Goombas (x5 Combo Hit!).
[Player] Combo hit count: 5. Publishing event...
[EventBus] Publishing: ComboHit (data: 5)
[StatisticsTracker]  Received ComboHit. Max combo: 5
[StatisticsTracker]  Tracking combat metrics...

>> Player collects 99 gold coins (Milestone!).
[Player] Collected 99 coins! Publishing event...
[EventBus] Publishing: CoinCollected (data: 99)
[StatisticsTracker]  Received CoinCollected. Total coins: 100
[AchievementManager]  Received CoinCollected. Evaluating achievements...
[AchievementManager]  *** UNLOCKED ACHIEVEMENT: Golden Hoarder! ***
[EventBus] Publishing: AchievementUnlocked (data: Golden Hoarder)

>> Player falls into pit.
[Player] Died! Publishing event...
[EventBus] Publishing: PlayerDied (data: Player*)
[StatisticsTracker]  Received PlayerDied. Death count: 1
============================================================
```

Figure 9: Console Output Simulation of EventBus Event-Driven Mediation

11 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?

   A) Creational

   B) Structural

   C) Behavioral

D) Architectural

2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new subscribers?

   A) Liskov Substitution Principle

   B) Single Responsibility Principle

   C) Interface Segregation Principle

   D) Open-Closed Principle

3. What is the "Lapsed Listener" problem in the Observer pattern?

   A) The subject stops listening to updates.

   B) Observers are not notified because the subject crashed.

   C) An observer is deleted without unsubscribing, causing memory leaks/crashes.

   D) Multiple subjects occupy the same listener list.

4. Which of the following is a key advantage of the Observer pattern?

   A) It ensures observers are updated sequentially.

   B) It decouples the subject from concrete observer details.

   C) It reduces memory overhead of notifications.

   D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

12 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- Refactoring.Guru. *Design Patterns: Observer*. https://refactoring.guru/design-patterns/observer